

Arithmetic Operations and Aggregate Functions

```
CREATE DATABASE BasicCalcDB;  
USE BasicCalcDB;
```

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(50),  
    marks INT,  
    age INT  
);
```

```
INSERT INTO students VALUES
(1,'John',80,20),
(2,'Sara',65,22),
(3,'Mike',90,21),
(4,'Anna',70,23),
(5,'David',60,20);
```

Arithmetic Operations

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

Q1. Add 5 marks to each student.

```
// SELECT name,marks,(marks + 5)AS new marks FROM students;
```

Q2. Add age and marks together.

```
//SELECT name,marks,age,(marks + age) AS total_value FROM students;
```

Q3. Reduce 10 marks from each student.

```
//SELECT name,marks,(marks - 10) AS reduced marks FROM students;
```

Q4. Find the difference between marks and age.

```
//SELECT name,marks (marks - age) AS difference FROM students;
```

Q5. Multiply marks by 2.

```
//SELECT name,marks,(marks*2) AS double_marks FROM students;
```

Q6. Multiply age and marks.

```
//SELECT name,(age*marks) AS result FROM students;
```

Q7. Divide marks by 2.

```
//SELECT name,marks,(marks/2) AS half_marks FROM students;
```

Q8. Divide marks by age.

```
//SELECT name,marks,(marks/age) AS half_marks FROM students;
```

Aggregate Functions

Function	Meaning
COUNT()	Calculates the number of rows.
SUM()	Totals the values.
AVG()	Determines the average.
MAX()	Finds the highest value.
MIN()	Finds the lowest value.

Q1. Count total students.

```
//SELECT COUNT(*) AS total_students FROM students;
```

Q2. Count students who have marks above 70.

```
// SELECT COUNT(*) AS student_above_70  
FROM students  
WHERE marks > 70;
```

Q3. Find total marks of all students.

```
//SELECT SUM(marks) AS total_marks FROM students;
```

Q4. Find total age of students.

```
//SELECT SUM(age) AS total_age FROM students;
```

Q5. Find average marks.

```
//SELECT AVG(marks) AS avg_marks FROM students;
```

Q6. Find average age.

```
//SELECT AVG(age) AS avg_age FROM students;
```

Q7. Find highest marks.

```
//SELECT MAX(marks) AS highest_marks FROM students;
```

Q8. Find oldest student age.

```
//SELECT MAX(age) AS oldset_age FROM student;
```

Q9. Find lowest marks.

```
//SELECT MIN(marks) AS lowst_marks FROM student;
```

Q10. Find youngest student age.

```
//SELECT MIN(age) AS youngest_age FROM student;
```